OpenClaw with General Agents

Personal Agent Harnesses, Chapter 1/2 — Revision & Viva-Prep Cheatsheet (English + Roman Urdu)
Source: agentfactory.panaversity.org/docs/openclaw

EN: Turn OpenClaw into your Personal AI Employee -- an open-source assistant replying on WhatsApp/Telegram/Discord, installed/configured through your own general agent. The agent does the work; you only make the decisions only you can make.
Roman Urdu: OpenClaw ko apna Personal AI Employee banao -- open-source assistant jo WhatsApp/Telegram/Discord se reply karta, aapke apne general agent se install/configure hota. Agent kaam karta; aap sirf wo faislay lete jo sirf aap le sakte.

1. The Collaboration Pattern (repeats every scenario)

- **5-step rhythm:** paste one sentence -> agent consults AGENTS.md and proposes a plan -> you approve and watch -> agent stops at the SEAM (only you can do this -- QR scan, API key, OAuth) -> done when something observable happens.

2. Install, Pairing, and the Agent Loop

- Install gives you AGENTS.md (~600-line operational reference). Everything lives under ~/.openclaw/ (config, key, workspace/brain).
- Phone pairing (WhatsApp/Telegram/Discord) makes your phone an AUTHENTICATED PATH -- treat it like a credential.
- **The agent loop (6-line shape):** inbound message -> model call -> tool call -> tool result -> second model call -> outbound message. Every later scenario just adds tools/triggers to this same loop.

3. Memory, Skills, and Automation

- Behavior lives in workspace markdown: SOUL.md (tone), IDENTITY.md (name/role), USER.md (about you), MEMORY.md (durable, cross-channel facts). Edit then /reset, or the old system prompt stays cached.
- Skill = folder + SKILL.md the agent invokes itself; MCP tool = an external service capability. 'Activation Dance': exists -> disabled by default -> enabled -> configured (restart).
- **Monthly audit** is the real defense -- an AI Employee quietly accumulates skills/credentials/tools; review on a fixed cadence, not just at install time.

4. Sandboxing with NemoClaw

- Pairing controls WHO can message; sandboxing controls what it can do with what it reads -- you need both. NemoClaw runs OpenClaw inside an OpenShell cage, 4 layers: Foundation, Guard, Prison, Employee.

Key Terms Cheatsheet

One line each — English + Roman Urdu.

Term	English	Roman Urdu
Seam	A step only a human can do (QR scan, API key, OAuth) -- the agent always stops here.	Step jo sirf insaan kar sakta (QR scan, API key, OAuth) -- agent hamesha yahan rukta.
Workspace files	SOUL.md, IDENTITY.md, USER.md, MEMORY.md -- OpenClaw's editable identity.	SOUL.md, IDENTITY.md, USER.md, MEMORY.md -- OpenClaw ki editable identity.
Activation Dance	Exists -> disabled by default -> enabled -> configured (restart).	Exists -> disabled by default -> enabled -> configured (restart).
Monthly audit	A fixed-cadence review that catches quietly accumulated risk.	Fixed-cadence review jo quietly accumulated risk pakarta.
NemoClaw sandbox	Runs OpenClaw inside a locked cage so a compromised message can't reach the whole computer.	OpenClaw ko locked cage mein chalata taake compromised message poora computer na reach kar sake.

Quick Self-Check

Question	Answer
What's the 5-step collaboration pattern that repeats across every OpenClaw scenario?	Paste a sentence, agent consults AGENTS.md and proposes a plan, you approve, agent stops at the seam, done when something observable happens.
Why must you /reset after editing a workspace file like SOUL.md?	Otherwise the old, cached system prompt stays in effect and your edit has no visible effect.
Why is phone pairing treated as a credential, not just a convenience?	Once paired, the phone is an authenticated path into the agent -- losing control of it is like losing an API key.
What's the real defense against an AI Employee's risk compounding over time?	A fixed-cadence (monthly) audit -- not vigilance only at install time, since each addition is a small approved decision that quietly compounds.
Why does OpenClaw need sandboxing in addition to pairing?	Pairing controls who can message it; sandboxing controls what it can do with what it reads -- prompt injection from a stranger's message needs the second layer.

Where This Leads / Sources

Leads to: Hermes (the depth-focused sibling), Governance Risk & Responsible Use. Full 8 scenarios + practice drills in the chapter's numbered files.

Source: docs/openclaw/ (this repo), content originally via Zia Tutor AI connector.